

Fern: offline flight study guide

Goal: arrive able to explain one complete Fern run, its important safety choices, and its current limitations. This guide is designed to be read without the repository or an internet connection.

How to use this guide

Do not try to memorise package names or read every implementation detail. For each section, pause and explain the idea aloud in your own words. Write down questions rather than getting stuck on a detail. The goal is a coherent mental model, not total source-code recall.

The codebase is roughly 33,000 lines of Go production code plus 16,700 lines of Go tests. Its scope includes a CLI/server, durable state, Docker lifecycle, OpenCode attachment, GitHub App credentials, artifacts, pairing, backup, and release tooling. A complete reading is not a sensible flight goal.

Part 1 - The product in plain English

Fern is a self-hosted control plane for coding-agent tasks. A user submits an instruction against one exact Git revision. Fern records that request durably, creates a disposable workspace and OpenCode container, lets the user inspect the live agent, then either discards the run or retains its final Git work.

The key distinction is this: Fern is not mainly an agent or an IDE. OpenCode does the reasoning, edits files, runs commands, and may push a branch or open a PR. Fern controls the lifecycle around that agent so it can answer questions such as: which exact container wrote this work? Did a request happen before the network response was lost? Is it safe to retry? Can the container now be removed without losing the result?

The durable product is a run record and, after sealing, a retained Git artifact. The Docker workspace is deliberately temporary.

A useful interview answer

"Fern coordinates long-running coding-agent work. It records intent before external effects, binds work to an exact container and Git revision, supports live attachment without adding a second writer, and can seal a Git artifact before it removes the disposable workspace."

Part 2 - What Fern owns, and what it does not

Fern owns:

- task admission, idempotency, run state, and recovery evidence;
- the exact repository/revision, container, volume, and session identities;
- short-lived repository-scoped GitHub App credentials;
- live-session attachment policy;
- sealing, Git-bundle retention, and cleanup;
- device pairing, plugin authorization, backup, and credential rotation.

OpenCode owns:

- the model conversation and provider interaction;
- edits, tool use, tests, questions, and permissions;

- Git pushes and draft PR creation when the task requests them.

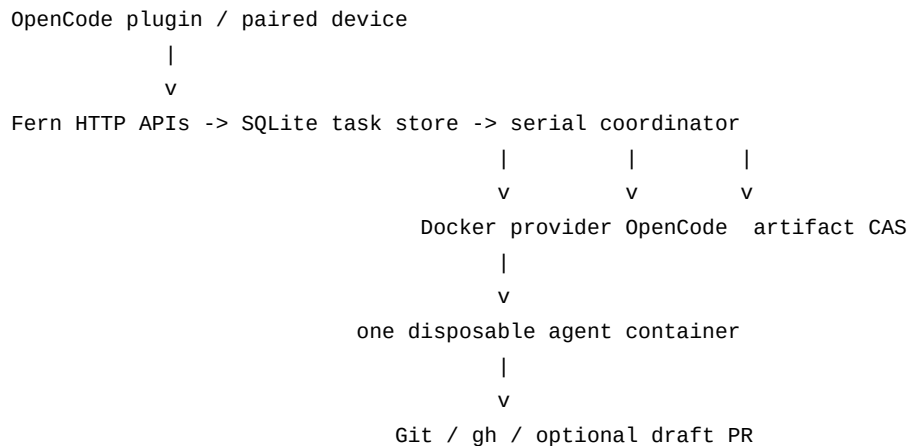
Fern does not independently prove that repository tests passed. It does not guarantee exactly-once PR creation. A PR can differ from the final sealed artifact if the agent edits again after pushing.

Part 3 - The normal run story

Read this as a narrative. It is the single most important thing to understand.

1. A user starts OpenCode in a clean, committed repository and selects /fern then Run.
2. The Fern plugin collects the canonical Git remote and exact HEAD, asks for confirmation, and sends a create request to Fern.
3. Fern authenticates the actor, validates the repository/revision against its bound host repository, and commits run intent to SQLite. A matching repeated request can return its original result through an idempotency key.
4. The serial coordinator wakes only after that database transaction commits.
5. The coordinator creates and attests an isolated clone, a private volume, and an exact Docker container using the qualified image and policy.
6. It starts the container, checks its identity and health, creates/reconciles the one OpenCode session, then records prompt intent before dispatching the instruction.
7. While the run works, Fern observes bounded OpenCode status/question signals and Docker usage. The run can be working or needs_you.
8. A user can attach with the normal OpenCode TUI. Attachment reaches the exact existing session; it is not another agent and does not own the filesystem.
9. The user chooses Stop or Seal. Both fence the exact writer. Seal then exports and verifies the Git result; Stop does not promise retention.
10. After a successful seal, Fern retains an artifact in local content-addressed storage, commits the result record, then removes the route, container, volume, and clone.

The simplified topology



Part 4 - The concepts that make Fern more than docker run

Durable intent before external effect

Network calls and Docker operations are not database transactions. A process may crash after sending a request but before receiving a response. Fern records an intent or started phase before it makes the corresponding external mutation.

After a restart it can inspect what exists and decide whether an operation actually happened.

This is why a task request, a prompt request, a seal request, and cleanup steps have durable records. Fern avoids treating a timeout as proof that nothing happened.

Idempotency

An idempotency key lets a client retry the *same* create request without creating duplicate work. Fern binds the key to the actor and canonical request hash. Reusing it for a different request conflicts instead of silently doing the wrong thing.

Idempotency is not magic. It does not prove an arbitrary external API supplied exactly-once behavior. Prompt dispatch, for example, has a no-replay fence.

Exact writer identity

A container name is not enough: names can be reused. Fern records a resource identity and an exact runtime identity that includes the Docker container ID and its canonical start time. Destructive operations re-inspect the resources and reject replacements or ambiguous ownership.

This supports the claim "stop this exact writer" rather than "stop whichever container currently has a familiar name."

State and phase are different

The visible state describes the user-level situation:

```

queued -> setting_up -> working <-> needs_you
      |               |
      +--> canceling / uncertain / result_ready / failed
              / cleanup_required

```

The durable phase records finer-grained effect boundaries, such as `provision_intent`, `container_observed`, `prompt_intent`, `writer_inactive`, `exporting`, `artifact_committed`, and `cleanup_complete`.

The distinction matters during restart recovery. "Working" is too vague to tell Fern whether a container existed, whether a prompt may have been sent, or whether artifact export had started.

Part 5 - Stop, Seal, result, and recovery

Stop means cancel and clean up. It is not a request to preserve unfinished work.

Seal is explicit and irreversible. Fern first stops and proves inactive the exact writer. It then captures committed, staged, unstaged, and untracked Git changes, creates a verified Git bundle and manifest, stores it in an artifact-CAS, materializes a detached checkout to verify it, commits the result tuple to SQLite, and only then removes disposable resources.

The phrase "verified result" means artifact bytes and identity agree. It does not mean tests passed, the code is correct, or a PR was created.

Recovery follows a conservative rule: retry an external mutation only when evidence proves it was not attempted. If Fern cannot prove the outcome, it keeps the run uncertain rather than blindly replaying a potentially

side-effecting action. Cleanup stays cleanup_required until absence is proven.

Interview answer: why not retry automatically?

"A lost response is ambiguous. The operation could have succeeded remotely even though the client did not receive the response. Replaying a prompt, a push, or a destructive cleanup action could duplicate work or affect the wrong resource, so Fern persists intent and reconciles evidence first."

Part 6 - Docker and security: be precise

Fern configures a container with a read-only root filesystem, bounded tmpfs writes, memory/CPU/PID limits, dropped capabilities, no-new-privileges, a seccomp profile, private IPC/cgroup namespaces, a loopback-only server port, and a qualified image. Its runtime storage is intended to be bounded by Linux XFS project byte and inode quotas.

Those are useful resource and lifecycle boundaries. They do not make Fern a safe hostile-code sandbox:

- bridge network egress is unrestricted;
- repository code and an agent may misuse or exfiltrate their short-lived repository token while it is valid;
- Docker administrators and the host administrator are trusted;
- Docker alone does not prove filesystem or network isolation.

The code intentionally rejects arbitrary environment injection, including a normal paid model-provider API key, because unrestricted network egress would make that credential unsafe under this threat model.

Part 7 - Credentials and access

Fern uses a GitHub App. Its private key stays on the Fern host. Fern mints a short-lived installation token restricted to the one bound repository and puts the rotating token in private runtime storage. Image-installed Git and gh helpers read it there.

The token is not intentionally put in Docker environment/configuration, command arguments, labels, logs, repository files, or retained artifacts. This reduces accidental secret persistence. It does not stop malicious code inside the agent container from reading its own token and sending it over the network.

There are three actor types at the HTTP edge:

- the operator, using loopback Basic authentication;
- a paired device, using a durable browser cookie;
- the OpenCode plugin, using a device-authorized fixed-scope bearer token.

The phone pairing flow is browser pairing over private Tailscale HTTPS. It is not a native mobile application and not yet a rich run dashboard.

Part 8 - Why the real demo is currently difficult

Fern has intentionally high execution requirements:

- native Linux (not macOS or Docker Desktop);
- a same-host local Docker socket;
- an XFS project-quota runtime filesystem with enforced byte and inode limits;
- a deliberate solution to Linux quota-query CAP_SYS_ADMIN requirements;
- a qualified image, bound Git repository, private HTTPS routes, and GitHub

App;

- a usable credential-free model provider.

The current repository passes its Go unit/integration tests, but the docs are honest that the macOS development host has not qualified actual XFS quota enforcement, a physical phone test, a host reboot, or a live GitHub mutation.

This is the right interview framing: Fern is a pre-release systems prototype with strong tested control-plane mechanics, not a finished hosted product or proven security sandbox.

Part 9 - Package map: only remember the roles

- cmd/fern: CLI commands, startup, dependency assembly, shutdown.
- run: lifecycle vocabulary and exact resource/runtime identity; no I/O.
- runcommand: create/stop/seal policy, idempotency, admission.
- runapi: plugin-authenticated HTTP run API.
- taskstore: SQLite durable state, receipts, claims, transitions.
- backgroundruncoord: serial engine that converts durable phases into effects.
- taskenvdocker: Docker clone/volume/container policy and credential handoff.
- backgroundopcode: pinned OpenCode session/prompt/observation protocol.
- backgroundroute: exact live-session attachment route and fencing.
- taskartifact: Git snapshot/bundle/manifest/CAS verification.
- taskresultsource: matches a durable result to freshly verified artifact bytes.
- proxy, control, pluginauth: HTTP ingress, browser pairing, device/plugin auth.
- githubapp: GitHub App onboarding and scoped token issuance.

The dependency direction is intentional: policy (run) does not import Docker, HTTP, the coordinator, or SQLite. The coordinator orchestrates; the task store owns durable transition legality; the Docker provider owns external resources.

Part 10 - A 13-hour tablet plan

Hour 0-1: Read Parts 1-3. Explain the ten-step normal run story aloud.

Hour 1-2: Read Parts 4-5. Write your own definitions for durable intent, idempotency, exact writer, phase, Stop, Seal, and uncertain.

Hour 2-3: Read Parts 6-8. Make a two-column list: "Fern protects against" and "Fern does not protect against."

Hour 3-4: Read Part 9 twice. Draw the package arrows from memory.

Hour 4-6: Re-read the normal run story and pretend you are the coordinator. For each step, say what durable evidence should exist before and after it.

Hour 6-7: Walk through a failure: prompt request times out after being sent. What must Fern avoid doing? What evidence can it inspect?

Hour 7-8: Walk through Seal. Why must the exact writer be stopped before export? Why must materialization be checked before cleanup?

Hour 8-9: Walk through credentials. What does the container receive? What never leaves the host? What risk remains?

Hour 9-10: Study the phone demo story: Tailscale route, one-time pairing code, paired browser, and its limits.

Hour 10-11: Prepare a two-minute verbal project explanation and repeat it until it is natural.

Hour 11-12: Prepare answers to the interview questions below.

Hour 12-13: Write a one-page plan for your first real demo: Linux host, model-provider decision, tiny blog task, evidence to capture, and limitation to state honestly.

Part 11 - Interview practice questions

1. Why does Fern exist if Docker can already run a coding agent?
2. What exactly becomes durable when a task is admitted?
3. Why is an idempotency key useful, and what does it not solve?
4. Why are container names insufficient for safe cleanup?
5. What is the difference between a user-visible state and a durable phase?
6. Explain Stop versus Seal without using jargon.
7. What does Fern verify about a sealed artifact, and what does it not verify?
8. Why can a PR differ from the sealed result?
9. Why does Fern report uncertainty instead of replaying every failed request?
10. Describe the container's security boundary honestly.
11. How are GitHub credentials delivered, and what is the remaining risk?
12. What are the main deployment blockers today?

Final one-minute summary

"Fern is a single-host control plane for coding-agent jobs. A plugin submits a task for one clean Git revision, and Fern records that intent in SQLite before it provisions a disposable Docker workspace. It binds the task to exact container and session identities, supplies short-lived repository-scoped GitHub credentials, and allows a user to attach to the existing agent session. When I seal a run, Fern fences the exact writer, exports and verifies the Git artifact, records the retained result, and then cleans up the workspace. It is not a hostile-code sandbox and it does not prove test success or exactly-once PRs. The current project is a pre-release prototype whose strict Linux/XFS and model-provider requirements still need a real end-to-end deployment."